Primer Proyecto: Servidor de simulaciones electromagnéticas FDTD concurrente

CI-0117 - Programación Paralela y Concurrente

Universidad de Costa Rica

Andrés Oreamuno Varela
Andrés Villegas Baltodano
Emanuel González Chaves

Octubre 2026

1. Introducción

El proyecto implementa un servidor escrito en C que recibe solicitudes de simulaciones FDTD de múltiples clientes vía TCP y las procesa mediante un solucionador paralelizado con Pthreads. Se utilizan diversos mecanismos de sincronización vistos en el curso (mutex, semáforos, barreras) y se pretende analizar el rendimiento utilizando la ley de Amdahl.

2. Desarrollo

El trabajo consta de tres ejecutables: fdtd_server, fdtd_client y fdtd_bench. El servidor se encuentra escuchando peticiones de parte de los clientes vía línea de comandos y las responde.

Para montar el servidor se utilizó como base el proyecto mini_server facilitado por el profesor, así como las tareas realizadas relacionadas con semáforos y mutex.

Se implementó una cola circular de capacidad 8, protegida por 1 mutex y 2 semáforos: el mutex asegura los accesos de lectura y escritura mientras que los semáforos indican cuantos campos libres y ocupados existen. En esta cola se implementa el patrón productor consumidor, el consumidor es un único hilo despachador.

Para cumplir con el enunuciado del proyecto se utilizó la función sem_trywait sobre el semáforo de campos libres, así si la cola esta llena el trabajo no se queda esperando y retorna BUSY, evitando que el cliente se bloquee y garantizando así no se pierda el trabajo.

El proyecto implementa un apagado ordenado, primero deja de aceptar conexiones, espera a que terminen los trabajos que los manejadores estan procesando y limpia la cola (los trabajos en espera se descartan). Finalmente libera los recursos.

3. Pruebas

Se realizaron pruebas manuales sobre el cliente y el servidor para corroborar el correcto funcionamiento de las funciones SUBMIT, STATUS Y SHUTDOWN. No fue posible terminar con la implementación del PEEK y el RESULT, ni con la unidad bench para automatizar las pruebas.

4. Conclusiones

No fue posible llegar a conclusiones basadas en métricas o aplicar la ley de Amhdal porque la unidad bench no se termino a tiempo.

Como conclusión podemos destacar la complejidad que se va evidenciando en proyectos de programación paralela conforme estos van creciendo en tamaño,

y la importancia del orden en el manejo de los recursos para evitar fugas de memoria o comportamientos incorrectos del sistema.

5. Repositorio

`https://github.com/andresoreamuno/fdtd-server`

Referencias

[1] A. B. Downey, *The Little Book of Semaphores*, 2.ª ed. Green Tea Press, 2016. [En línea]. Disponible: `https://greenteapress.com/wp/semaphores/`

[2] P. S. Pacheco, *An Introduction to Parallel Programming*. Burlington, MA, EE. UU.: Morgan Kaufmann, 2011.